

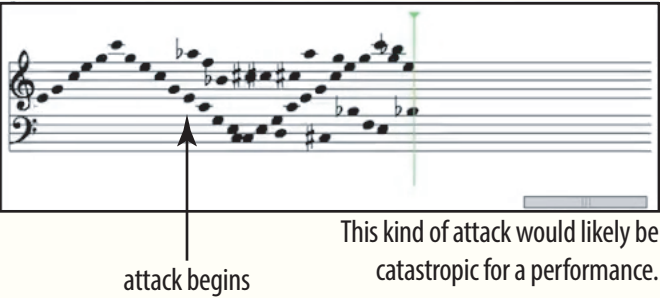
Securing OSC (Open Sound Control) over UDP

paul v. miller // computer security 2026

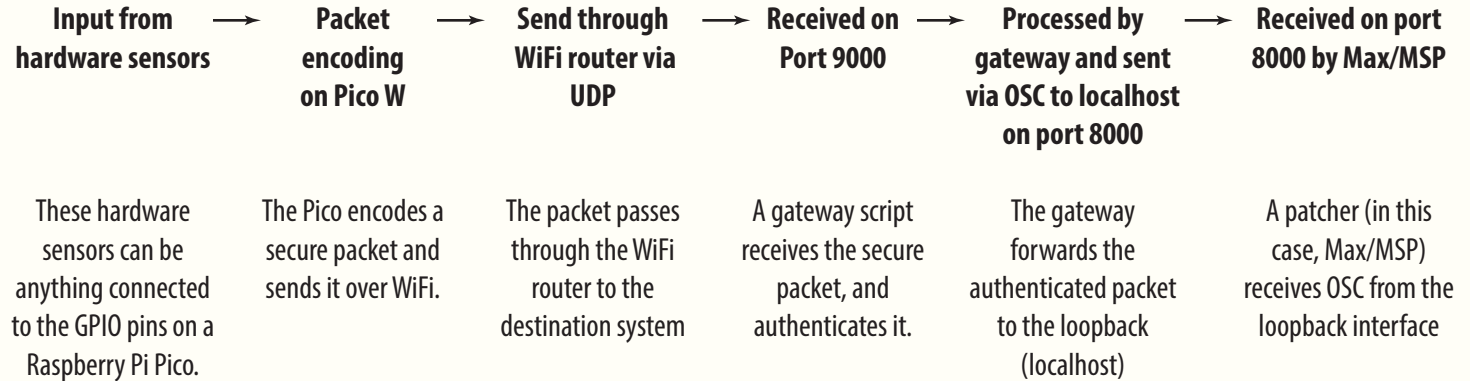
1.1 The Problem or Challenge

This project presents a lightweight authentication layer for transmitting OSC (Open Sound Control) packets over UDP, using HMAC-SHA256 (Hash Message Authentication Code + a cryptographic hash function) and sequence-based replay protection. It is designed for resource-constrained embedded controllers that operate in live performance or real-time musical applications.

An attack on OSC: A victim transmitting OSC code that translates to a 3-octave C major arpeggio is attacked by a second device on the same network and port, transmitting random notes.



1.2 Secure System Overview



3. Threat Model and Attack Resilience

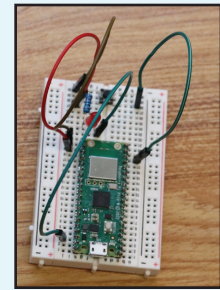
For an attack to occur, the malevolent agent must be on the same WiFi network and know the port that the victim is using. It is possible to scan all 65,536 ports in a few seconds; any successful hit will probably be heard. Also due to the prevalence of online tutorials, a few ports (7000, 8000, 9000) are very frequently used for OSC transmission over UDP. Therefore a successful attack is most likely going to happen from an insider. While it is not very likely, the results of an attack would be catastrophic, possibly leading to a total collapse of the performance and significant reputational harm. I subjected the secure system to three attacks from a separate Raspberry Pi Pico W.

3.1 First Attack

The attacker sent regular OSC packets with no header or HMAC tag. The secure system rejected these and outputted the message DROP bad_length to stdio. Bad packets were not forwarded to the loopback.

3.2. Second Attack

The attacker sent a packet with the right structure (header, payload, HMAC tag) but the HMAC tag was generated with the wrong key. The secure system rejected the bad packets and responded with the output "DROP bad_hmac" to stdio. No bad packets were forwarded to the loopback.



Attacker Raspberry Pi Pico W: One button (on/off) and red status LED

3.3 Third Attack

The attacker somehow captured an authentic packet with the correct structure and properly generated HMAC tag. In this case, I simply replayed packets that the system received, using a diagnostic script I wrote. A better hacker could potentially acquire packets by sniffing them. The secure system rejected the injected packet because its monotonic sequence number was out of order, printing the message "DROP replay" to stdio. Again, no bad packets were forwarded to the loopback.

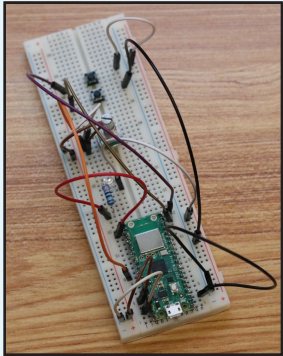
2. Secure Packet Design

On the secure system, the Raspberry Pi Pico W runs MicroPython code that reads values from a minimal musical control system consisting of a 10kΩ potentiometer and two buttons. It creates a 12-byte header, then appends the OSC string, and finally adds a 16-byte truncated HMAC-SHA256 authentication tag. It sends this packet through UDP over WiFi to the destination IP address at port 9000. The secure packet is constructed like this:



On the receiving end, a gateway receives the packet on port 9000, authenticates it, and then forwards it to the loopback (127.0.0.1) on port 8000. From here, Max/MSP (or, for that matter, anything else on the system that is listening on port 8000) can get the authenticated OSC packet.

Secure system: Raspberry Pi Pico W with potentiometer, white status LED and two 6×6mm buttons



4. Outcomes and Results

The system survived all three attacks, while providing *integrity* through HMAC and *authenticity* through a shared key. There was no *confidentiality* assurance as the system did not use any data encryption: a sacrifice that was made in order to ensure speed.

I measured system latency in two ways. First, I calculated round-trip packet time as there is no way to synchronize clocks on the Pico and the receiver. These data include WiFi transit time, OS scheduling and possible buffer delays. A more accurate measure of the latency that the system introduces is gateway processing time. This is the time it takes to process the secure OSC packets. Gateway processing time is extremely low, averaging **58 μs**:

Measurement	Range	Mean	Outliers
Plain OSC: Round-Trip Time	211.1 – 211.5 ms	211.5 ms	310.8 ms
Secure OSC: Round-Trip Time	211.2 – 212.2 ms	211.8 ms	512.8 ms
Plain OSC: Gateway Processing Time	97.2 – 210.0 μs	160.8 μs	none
Secure OSC: Gateway Processing Time	146.9 – 366.5 μs	218.6 μs	366.5 μs

difference:
58 μs

Latency Times as Measured over 30 Packets

Future versions of the system should at minimum include more flexibility in the length of the OSC payload. Other features that may be worth pursuing are data encryption (though this could add substantially to latency), and a sliding acceptance window that would accept packets if received slightly out of order. These options should probably not be hard-wired into the system, but rather options that the user could select themselves.